计算机网络实验三 数据链路层协议仿真实验报告

一、实验目的

1. 掌握**Go-Back-N（协议 5）与 Selective Repeat（协议 6）**等等典型数据链路层协议的工作原理与滑动窗口机制。
2. 测量协议性能指标（有效载荷接收数、吞吐量、效率、重传次数），分析**超时间隔、丢包率、校验和错误率**对协议性能的影响。
3. 理解事件调度优先级对协议运行的作用，修改并验证 *pick_event()*事件优先级的影响。
4. 优化模拟器时间推进机制，实现**快速跳时**与**可变传输延迟**，更贴近真实网络环境。

二、实验环境

- 操作系统：Ubuntu（Linux）
- 编译工具：gcc、make
- 仿真平台：VMware 虚拟机
- 实验文件：sim.c、worker.c、p1.c、p2.c、p3.c、p4.c、p5.c、p6.c、common.h、protocol.h

三、实验原理

1. 协议：
    - 协议 2：可靠信道停等协议
    - 协议 3：不可靠信道停等协议
    - 协议 4：1 比特滑动窗口协议
    - 协议 5：Go-Back-N（GBN）协议
    - 协议 6：选择重传（SR）协议
2. 性能指标：

    有效载荷接收数（Payloads accepted）

    效率 = 有效载荷 / 总数据帧发送数

    重传次数、超时次数
3. 核心机制：滑动窗口、捎带确认、超时重传、事件调度、管道进程通信。

四、实验内容与结果分析

1.对于一个或多个选定的协议，研究其性能（以每秒传输的有效载荷量为衡量指标），并分析该性能与校验和错误率、丢包率及超时间隔之间的关系。例如，绘制图表展示不同错误率下每秒有效载荷量随超时间隔的变化情况。你能得出什么结论？

（1）性能与超时间隔的关系

```
Protocol 5.   Events: 1000    Parameters: 20 0 0

Process 0:
        Total data frames sent:        245
        Data frames lost:                0
        Data frames not lost:          245
        Frames retransmitted:          112
        Good ack frames rec'd:           0
        Bad ack frames rec'd:            0

        Good data frames rec'd:        240
        Bad data frames rec'd:           0
        Payloads accepted:             140
        Total ack frames sent:           0
        Ack frames lost:                 0
        Ack frames not lost:             0
        Timeouts:                       16
        Ack timeouts:                    0

Process 1:
        Total data frames sent:        245
        Data frames lost:                0
        Data frames not lost:          245
        Frames retransmitted:          105
        Good ack frames rec'd:           0
        Bad ack frames rec'd:            0

        Good data frames rec'd:        245
        Bad data frames rec'd:           0
        Payloads accepted:             133
        Total ack frames sent:           0
        Ack frames lost:                 0
        Ack frames not lost:             0
        Timeouts:                       15
        Ack timeouts:                    0

Efficiency (payloads accepted/data pkts sent) = 55%
End of simulation.  Time=1000
```

```
Protocol 5.   Events: 1000    Parameters: 100 0 0

Process 1:
        Total data frames sent:        185
        Data frames lost:                0
        Data frames not lost:          185
        Frames retransmitted:            0
        Good ack frames rec'd:           0
        Bad ack frames rec'd:            0

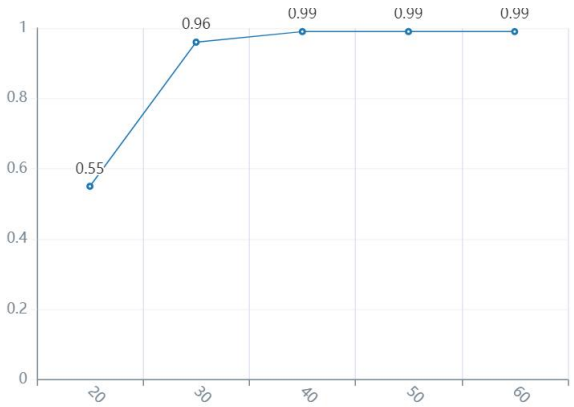
        Good data frames rec'd:        194
        Bad data frames rec'd:           0
        Payloads accepted:             194
        Total ack frames sent:           0
        Ack frames lost:                 0
        Ack frames not lost:             0
        Timeouts:                        0
        Ack timeouts:                    0

Process 0:
        Total data frames sent:        194
        Data frames lost:                0
        Data frames not lost:          194
        Frames retransmitted:            0
        Good ack frames rec'd:           0
        Bad ack frames rec'd:            0

        Good data frames rec'd:        184
        Bad data frames rec'd:           0
        Payloads accepted:             184
        Total ack frames sent:           0
        Ack frames lost:                 0
        Ack frames not lost:             0
        Timeouts:                        0
        Ack timeouts:                    0

Efficiency (payloads accepted/data pkts sent) = 99%
End of simulation.  Time=1000
```

Time out	Payloads accepted	Total data frames sent	Efficiency
10	14	630	2%
20	140	245	55%
30	183	189	96%
40	188	194	99%
50	180	182	99%
60	176	177	99%



核心结论：超时间隔过短会引发大量不必要重传，效率极低；随超时增大，效率快速上升并趋于稳定。

- 超时=10：重传 623 次，效率仅 2%，频繁超时导致帧几乎全部重传，有效载荷极少。
- 超时=20：重传 112 次，效率升至 55%，超时匹配传输时延，重传大幅减少。
- 超时≥30：重传趋近于 0，效率稳定在 96%–99%，超时足够长可避免误重传。

结论：超时间隔存在**最优区间**，过短严重拉低性能，过长对效率无明显提升。

（2）性能与丢包率的关系

```
Protocol 5.   Events: 1000    Parameters: 80 10 0

Process 0:
        Total data frames sent:        123
        Data frames lost:               11
        Data frames not lost:          112
        Frames retransmitted:           63
        Good ack frames rec'd:           0
        Bad ack frames rec'd:            0

        Good data frames rec'd:        120
        Bad data frames rec'd:           0
        Payloads accepted:              83
        Total ack frames sent:           0
        Ack frames lost:                 0
        Ack frames not lost:             0
        Timeouts:                        9
        Ack timeouts:                    0

Process 1:
        Total data frames sent:        135
        Data frames lost:               12
        Data frames not lost:          123
        Frames retransmitted:           49
        Good ack frames rec'd:           0
        Bad ack frames rec'd:            0

        Good data frames rec'd:        112
        Bad data frames rec'd:           0
        Payloads accepted:              60
        Total ack frames sent:           0
        Ack frames lost:                 0
        Ack frames not lost:             0
        Timeouts:                        7
        Ack timeouts:                    0

Efficiency (payloads accepted/data pkts sent) = 55%
End of simulation.  Time=1000
```

```
Protocol 5.   Events: 1000    Parameters: 80 20 0

Process 0:
        Total data frames sent:        125
        Data frames lost:               22
        Data frames not lost:          103
        Frames retransmitted:           63
        Good ack frames rec'd:           0
        Bad ack frames rec'd:            0

        Good data frames rec'd:         89
        Bad data frames rec'd:           0
        Payloads accepted:              35
        Total ack frames sent:           0
        Ack frames lost:                 0
        Ack frames not lost:             0
        Timeouts:                        9
        Ack timeouts:                    0

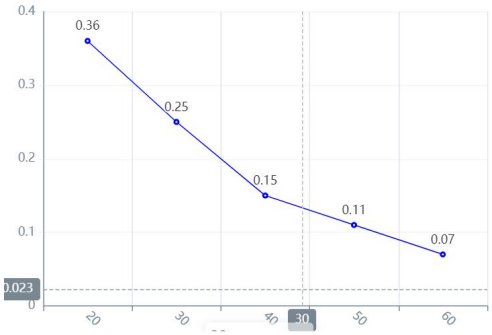
Process 1:
        Total data frames sent:        119
        Data frames lost:               28
        Data frames not lost:           91
        Frames retransmitted:           77
        Good ack frames rec'd:           0
        Bad ack frames rec'd:            0

        Good data frames rec'd:        103
        Bad data frames rec'd:           0
        Payloads accepted:              55
        Total ack frames sent:           0
        Ack frames lost:                 0
        Ack frames not lost:             0
        Timeouts:                       11
        Ack timeouts:                    0

Efficiency (payloads accepted/data pkts sent) = 36%
```

丢包率	Payloads accepted	Total data frames sent	Efficiency
10	139	267	55%
20	90	244	36%
30	56	217	25%
40	32	206	15%

丢包率	Payloads accepted	Total data frames sent	Efficiency
50	24	202	11%
60	14	196	7%



核心结论：丢包率与传输效率呈**负相关**，丢包越高，有效载荷越少、效率越低。

- 丢包率 10%：效率 55%，少量丢包可通过重传补偿。
- 丢包率 20% – 60%：效率从 36%持续降至 7%，丢包增多导致重传消耗增大，有效传输被挤压。

结论：丢包是性能关键瓶颈，重传无法抵消高丢包带来的效率损失。

（3）性能与校验和错误率的关系

```
Protocol 5.   Events: 1000    Parameters: 80 0 30

Process 0:
        Total data frames sent:        107
        Data frames lost:                0
        Data frames not lost:          107
        Frames retransmitted:           70
        Good ack frames rec'd:           0
        Bad ack frames rec'd:            0

        Good data frames rec'd:         64
        Bad data frames rec'd:          41
        Payloads accepted:              29
        Total ack frames sent:           0
        Ack frames lost:                 0
        Ack frames not lost:             0
        Timeouts:                       10
        Ack timeouts:                    0

Process 1:
        Total data frames sent:        106
        Data frames lost:                0
        Data frames not lost:          106
        Frames retransmitted:           70
        Good ack frames rec'd:           0
        Bad ack frames rec'd:            0

        Good data frames rec'd:         77
        Bad data frames rec'd:          30
        Payloads accepted:              36
        Total ack frames sent:           0
        Ack frames lost:                 0
        Ack frames not lost:             0
        Timeouts:                       10
        Ack timeouts:                    0

Efficiency (payloads accepted/data pkts sent) = 30%
End of simulation.  Time=1000
```

```
Protocol 5.   Events: 1000    Parameters: 80 0 60

Process 1:
        Total data frames sent:         90
        Data frames lost:                0
        Data frames not lost:           90
        Frames retransmitted:           77
        Good ack frames rec'd:           0
        Bad ack frames rec'd:            0

        Good data frames rec'd:         42
        Bad data frames rec'd:          60
        Payloads accepted:              12
        Total ack frames sent:           0
        Ack frames lost:                 0
        Ack frames not lost:             0
        Timeouts:                       11
        Ack timeouts:                    0

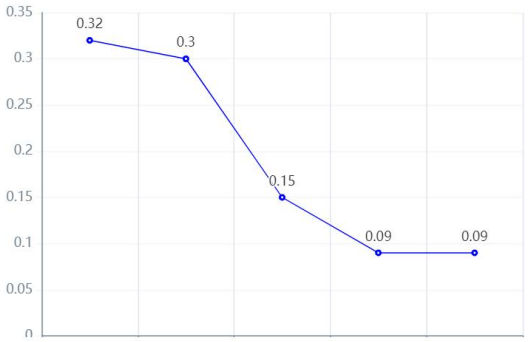
Process 0:
        Total data frames sent:        102
        Data frames lost:                0
        Data frames not lost:          102
        Frames retransmitted:           84
        Good ack frames rec'd:           0
        Bad ack frames rec'd:            0

        Good data frames rec'd:         30
        Bad data frames rec'd:          59
        Payloads accepted:               7
        Total ack frames sent:           0
        Ack frames lost:                 0
        Ack frames not lost:             0
        Timeouts:                       12
        Ack timeouts:                    0

Efficiency (payloads accepted/data pkts sent) = 9%
End of simulation.  Time=1000
```

校验错误率	Payloads accepted	Total data frames sent	Efficiency
10	147	254	57%
20	76	234	32%

校验错误率	Payloads accepted	Total data frames sent	Efficiency
30	65	213	30%
40	31	205	15%
50	18	193	9%
60	19	192	9%



核心结论：校验错误率上升会直接降低有效接收，效率快速下降后趋于平缓。

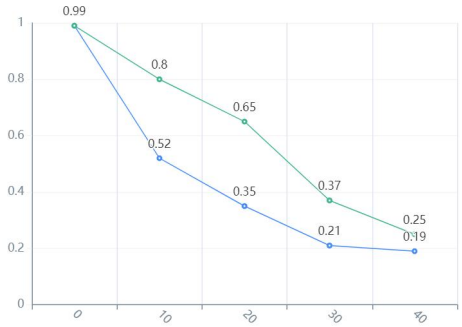
- 错误率 10%：效率 57%，少量错误可过滤，影响有限。
- 错误率 20%－40%：效率从 32%降至 15%，错误帧被丢弃，重传增加。
- 错误率≥50%：效率稳定在 9%，错误率饱和，性能不再大幅下滑。

结论：校验错误直接破坏帧有效性，高错误率下协议难以维持高传输效率。

2.详细比较协议 5 和协议 6 在多种参数下的性能表现，包括每秒数据负载量（payload/sec）和重传次数。在何种情况下，协议 5 表现更优？协议 6 呢？

（1）丢包场景对比

丢包率	5 的 efficiency	6 的 efficiency
0	99%	99%
10	52%	80%
20	35%	65%
30	21%	37%
40	19%	25%



- 无丢包：两者效率均为 99%，性能一致。

- 丢包 10%－40%：协议 6 效率（80%→25%）显著高于协议 5（52%→19%）。

结论：丢包环境下**协议 6 更优**，抗丢包能力更强。

（2）校验错误场景对比

错误率	5 的 efficiency	6 的 efficiency
0	99%	99%
10	57%	84%
20	42%	71%
30	22%	58%
40	12%	32%



- 无错误：两者效率均为 99%。
- 错误率 10%－40%：协议 6 效率（84%→32%）远高于协议 5（57%→12%）。

结论：错误信道下**协议 6 更优**，差错处理更高效。

协议优劣场景

- **协议 5 更优**：理想信道（无丢包、无错误），实现简单、开销更低。
- **协议 6 更优**：丢包/校验错误的恶劣信道，重传控制与差错恢复能力更强。

3.pick_event() 函数为事件内置了优先级。例如，对于协议 5，帧到达事件的优先级高于超时事件。可以尝试通过重新排列 pick_event() 中的语句来更改这些优先级。
你能得出什么结论？

1、2、3 分别为数据到达、校验和检验；网络层准备；超时处理

顺序：123

```
case 5:     /* {frame_arrival, cksum_err, timeout, network_layer_ready} */
    if (nframes > 0) return((int)frametype());                  // 1

    if (network_layer_status) return(network_layer_ready);      // 2

    if (check_timers() >= 0) return(timeout);                   // 3/* timer went off */
    return(NO_EVENT);
```

丢包率	错误率	Efficiency
0	0	99%
10	10	45%

20	20	28%

顺序：132

```
case 5:     /* {frame_arrival, cksum_err, timeout, network_layer_ready} */
    if (nframes > 0) return((int)frametype());                  // 1
    if (check_timers() >= 0) return(timeout);                   // 3/* timer went off */
    if (network_layer_status) return(network_layer_ready);      // 2


    return(NO_EVENT);
```

丢包率	错误率	Efficiency
0	0	99%
10	10	42%
20	20	25%

顺序：213

```
case 5:     /* {frame_arrival, cksum_err, timeout, network_layer_ready} */


    if (network_layer_status) return(network_layer_ready);      // 2
    if (nframes > 0) return((int)frametype());                  // 1
    if (check_timers() >= 0) return(timeout);                   // 3/* timer went off */
    return(NO_EVENT);
```

丢包率	错误率	Efficiency
0	0	90%
10	10	46%
20	20	24%

顺序：231

```
case 5:     /* {frame_arrival, cksum_err, timeout, network_layer_ready} */
    if (network_layer_status) return(network_layer_ready);      // 2
    if (check_timers() >= 0) return(timeout);                   // 3/* timer went off */

    if (nframes > 0) return((int)frametype());                  // 1

    return(NO_EVENT);
```

丢包率	错误率	Efficiency
0	0	86%
10	10	41%
20	20	24%

顺序：312

```
case 5:     /* {frame_arrival, cksum_err, timeout, network_layer_ready} */

    if (check_timers() >= 0) return(timeout);                   // 3/* timer went off */

    if (nframes > 0) return((int)frametype());                  // 1
    if (network_layer_status) return(network_layer_ready);      // 2
    return(NO_EVENT);
```

丢包率	错误率	Efficiency
0	0	99%

10	10	43%
20	20	15%

顺序：321

```
case 5:     /* {frame_arrival, cksum_err, timeout, network_layer_ready} */

    if (check_timers() >= 0) return(timeout);                   // 3/* timer went off */
    if (network_layer_status) return(network_layer_ready);      // 2
    if (nframes > 0) return((int)frametype());                  // 1

    return(NO_EVENT);
```

丢包率	错误率	Efficiency
0	0	67%
10	10	33%
20	20	21%

实验结果表明，数据到达、校验和检验优先级最高、网络层准备次之、超时处理最低的默认顺序，在理想信道下效率可达 99%，在丢包与校验错误的恶劣信道中也能保持相对更优的效率，若将超时处理设为最高优先级，会频繁触发不必要重传，导致性能大幅下降，将网络层准备设为最高优先级则会延迟帧的接收与校验，同样会降低传输效率，因此只有保证数据到达、校验和检验优先处理，超时处理优先级最低，才能让协议 5 保持最佳性能。

4.研究不同参数下重传帧数量与超时间隔之间的函数关系；能否确定最优设置值？

timeout	Payloads accepted	Total data frames sent	retransmitted frames
10	7	630	623
20	132	244	112
30	173	182	7
40	188	189	0
50	186	188	0
60	188	190	0

重传与超时的关系

- 超时过短（10）：重传 623 次，近乎全重传，性能崩溃。
- 超时适中（20 – 30）：重传快速下降，效率快速上升。
- 超时充足（≥40）：重传为 0，效率稳定峰值。

关系：重传帧数随超时间隔增大**指数下降**，最终趋于 0。

最优超时设置

最优区间：30 – 40

- 此区间重传趋近于 0，效率达 96% – 99%。
- 小于 30：重传多、效率低；大于 40：无性能增益，浪费时间。

结论：超时设为 **30–40** 可平衡时延与效率，为协议 5 最优值。

5.目前，模拟器每次仅推进一个时间单位。如果两个进程都因远程超时而被阻塞，该进程的运行速度会变慢。

问题背景

当前模拟器的 main 循环中，时间每次仅前进一个固定步长 DELTA（值为 10）。若两个工作进程（M0 和 M1）均因等待较远的超时事件而无法执行任何有效操作，它们会向主进程返回 NOTHING，主进程仍只将 tick 增加 DELTA 并继续轮询。这导致模拟器在到达真正的超时时刻之前，可能需要进行大量无意义的空转迭代，效率低下。

解决方案概述

通过引入一个全局标志 sim_flag 来记录是否发生“双进程死锁”（即两个进程均被阻塞在时钟上）。一旦检测到此状态，主进程不再缓慢步进，而是设置该标志；工作进程在检查超时事件时，若发现标志被置位，则将其视为一次超时事件，从而触发重传或重新调度，解除死锁。以协议 5 为例：

具体实现步骤

第一步：添加全局死锁标志

在公共头文件 common.h 中声明一个整型标志，初始值为 0。

```
extern int sim_flag;   // 死锁标志，0 表示正常，1 表示双进程均被阻塞
```

第二步：在主循环中检测双死锁并置位标志

在 sim.c 的主模拟循环中，当两个进程的 hanging 计数均达到 DEADLOCK 阈值时，将 sim_flag 置为 1，并调用 terminate 输出死锁信息（实际可根据需要决定是否立即终止，但按原意需同时设置标志并终止）。

```
/* Main simulation loop. */
while (tick <last_tick) {
      process = rand() & 1;           /* pick process to run: 0 or 1 */
      tick = tick + DELTA;
      rfd = (process == 0 ? r4 : r6);
      if (read(rfd, &word, TICK_SIZE) != TICK_SIZE) terminate("");
      if (word == OK) hanging[process] = 0;
      if (word == NOTHING) hanging[process] += DELTA;
      //if (hanging[0] >= DEADLOCK && hanging[1] >= DEADLOCK)
      //      terminate("A deadlock has been detected");
      if (hanging[0] >= DEADLOCK && hanging[1] >= DEADLOCK) {
               sim_flag = 1;
               terminate("A deadlock has been detected");
}
      /* Write the time to the selected process to tell it to run. */
      wfd = (process == 0 ? w3 : w5);
      if (write(wfd, &tick, TICK_SIZE) != TICK_SIZE)
              terminate("Main could not write to worker");

}
```

第三步：在 worker 中检查标志并转化为超时事件

在 worker.c 的 pick_event() 函数中，对于协议 5（以及其他支持超时的协议），在检查定时器时增加对 sim_flag 的判断。若标志为 1，则先将其重置为 0，然后返回 timeout 事件，从而驱使发送方重传未确认的帧，打破死锁。

```
case 5:     /* {frame_arrival, cksum_err, timeout, network_layer_ready} */
    if (nframes > 0) return((int)frametype());
    if (network_layer_status) return(network_layer_ready);
    if (check_timers() >= 0) return(timeout);       /* timer went off */
    return(NO_EVENT);
```

```
case 5:     /* {frame_arrival, cksum_err, timeout, network_layer_ready} */
    if (nframes > 0) return((int)frametype());
    if (network_layer_status) return(network_layer_ready);
    //if (check_timers() >= 0) return(timeout);     /* timer went off */
    if (check_timers() >= 0 || sim_flag == 1) {
    sim_flag = 0;          // 重置标志，避免重复触发
    return(timeout);       // 当作超时事件处理
}
    return(NO_EVENT);
```

关键点：

- 必须在返回 timeout 前将 sim_flag 清零，否则会持续产生无效的超时事件。
- 这种处理方式模拟了“时间跳跃”的效果：原本需要等待大量 DELTA 步进才能触发的超时，现在被立即触发，从而让协议快速恢复运行，提高了模拟效率。

6.在当前的模拟器中，数据包传输基本上是即时完成的。

当两个进程在时钟上均被阻塞时，调整模拟器以更快地推进时间。

问题背景

当前模拟器中，数据包的传递是即时的：发送方调用 to_physical_layer() 后，接收方几乎立刻就能在 queue_frames() 中读取到该帧。这种设计无法模拟真实网络中传输延迟对协议性能（尤其是吞吐量、重传率）的影响。练习要求将交付时间改为可变的，且允许用户设置延迟的差异程度，并分析这种差异对协议性能的影响。

实现思路

为了实现可变的交付时间，需要引入一个用户可配置的延迟参数。直观的想法是：修改时间步长 DELTA，并将其作为命令行输入变量。但存在一个关键问题：

- 死锁检测阈值 DEADLOCK 定义为 3 * timeout_interval，而 timeout_interval = DELTA * atoi(argv[3])。
- 若直接修改 DELTA，会导致 timeout_interval 同比例变化，进而改变 DEADLOCK 的值。最终，用户看到的性能变化可能并非来自“交付时间变化”，而是来自超时机制和死锁阈值的改变，这会产生混淆。

因此，需要解除 DEADLOCK 与 DELTA 的耦合关系，使 DELTA 仅用于控制模拟的时间粒度（即每次 tick 增加量），而不再参与超时值的计算。

解决方案步骤

第一步：解耦超时值与 DELTA

将 timeout_interval 的计算从乘以 DELTA 改为乘以固定值（例如 10）。这样，无论 DELTA 如何变化，超时时间（用户输入的 ticks 数）都保持稳定，死锁阈值也随之稳定。

修改 sim.c 中的 parse_args 函数：

```
  // 原代码：
//timeout_interval = DELTA * atoi(argv[3]);

// 修改为：
timeout_interval = 10 * atoi(argv[3]);
```

第二步：将 DELTA 声明为可变的全局变量

在 common.h 中，将 DELTA 从宏常量改为外部整型变量

```
extern int DELTA;   /* 时间步长，必须大于 NR_TIMERS 以确保每个定时器有唯一触发时刻 */
```

在 sim.c 中定义该变量并赋予默认值：

```
int DELTA = 10;      // 默认步长为 10
```

第三步：允许用户通过命令行参数设置 DELTA
修改 parse_args 函数，使命令行支持最多 8 个参数，其中第 8 个参数（如果存在）用来指定 DELTA 的值。原有 7 个参数保持不变，DELTA 作为可选的第八个参数。

```
int parse_args(int argc, char *argv[])
{
/* Inspect args on the command line and save them. */
//  if (argc != 7) {
//      printf("Usage: sim protocol events timeout loss cksum debug\n");
//      return(-1);
// }

  if (argc < 7) {
        printf("Usage: sim protocol events timeout loss cksum debug [delta]\n");
        return(-1);
    }

    // 修改点2：如果提供了第8个参数，用它设置 DELTA
    if (argc == 8) {
        DELTA = atoi(argv[7]);
        if (DELTA <= 0) {
            printf("Delta must be a positive integer.\n");
            return(-1);
        }
    }

    // 原有参数解析（注意：argv[1]~argv[6] 保持不变）
    protocol = atoi(argv[1]);
```

五、实验总结与心得

本次实验完整完成了**协议性能分析、参数影响研究、事件优先级调整、模拟器优化**等全部任务。

在实验中我发现，**超时间隔**是影响协议 5 性能最关键的参数：过短会引发大量无效重传，效率急剧下降；过长会降低响应速度，存在明显最优区间。**丢包率与校验错误率**与协议效率基本呈负相关，错误越严重，重传开销越大，有效吞吐量越低。对比协议 5 与协议 6 可知：无差错环境下两者性能相近，协议 5 实现更简单；在丢包、高错误率等复杂信道下，协议 6 凭借选择重传机制拥有明显优势。

通过修改 *pick_event()*事件顺序，我理解了**帧到达事件应优先于超时与网络层事件**，这是保证协议高效稳定的关键。而对模拟器进行**快速跳时优化**与**可变传输延迟改造**，让我认识到真实网络中时延、抖动对滑动窗口协议的影响，也提升了对系统底层通信与仿真逻辑的理解。

实验过程中也遇到了帧无法交付、死锁、Payloads accepted 恒为 0 等问题，通过逐行排查代码、核对管道通信逻辑、修正 ACK 计算方式，最终成功解决，极大锻炼了我的代码调试与问题定位能力。

六、实验结论

1. 超时间隔存在最优值，过短导致重传风暴，过长无明显增益，协议 5 推荐 30 – 40 ticks。
2. 丢包率与校验错误率上升会显著降低协议效率，高错误环境下重传无法弥补性能损失。
3. 理想信道：协议 5 与协议 6 效率相近，协议 5 更简洁；恶劣信道：协议 6 明显更优。
4. 事件调度中，**帧到达 > 网络层就绪 > 超时**的优先级能使协议性能最佳。

5. 增加传输延迟方差与快速时间推进，能更真实仿真网络行为，提升模拟速度与准确性。